ГУАП

КАФЕДРА № 43

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ

ПРЕПОДАВАТЕЛЬ

ассистент		А.Э.Зянчурин
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

РАЗРАБОТКА ТРЕБОВАНИЙ. МОДЕЛИРОВАНИЕ ПРЕДМЕТНОЙ ОБЛАСТИ. ПРОЕКТИРОВАНИЕ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ (Роль участника: Архитектор программного обеспечения)

по курсу: ОСНОВЫ ПРОГРАММНОЙ ИНЖЕНЕРИИ

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4431		Кочнев Д. О.
	подпись, дата	инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....3

ВВЕДЕНИЕ.....4

1 Архитектура программного обеспечения.....5

1.1 Исходные требования и архитектурные ограничения.....5

1.2 Структурные элементы ПО и их интерфейсы.....6

1.3 Компоненты и соединители во время выполнения.....10

1.4 Выбор архитектуры и архитектурные представления.....12

1.5 Структуры распределения и размещения.....13

1.6 Модель хранения данных и инварианты.....17

1.7 Сквозные сценарии и согласование транзакций.....18

1.8 Алгоритмы критических операций.....19

1.9 Контракты внешних интерфейсов.....20

1.10 Безопасность, надежность и эксплуатация.....21

1.11 Прослеживаемость и проверки.....22

1.12 Выводы по архитектуре.....22

2 Краткое описание хода выполнения.....23

ЗАКЛЮЧЕНИЕ.....24

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....25

ПРИЛОЖЕНИЕ А.....26

ПРИЛОЖЕНИЕ Б.....27

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Таблица 1 – Используемые сокращения

Сокращение	**Значение**
АИС	Автоматизированная информационная система
API	Программный интерфейс
REST	Стиль взаимодействия клиента и сервера через ресурсы
DTO	Объект передачи данных
ACID	Атомарность, согласованность, изоляция и долговечность транзакций
RBAC	Разграничение доступа по ролям
CI/CD	Непрерывная интеграция и доставка изменений
VPS	Виртуальный выделенный сервер
GHCR	Реестр контейнерных образов GitHub
TLS	Защищенный транспорт данных
FR / NFR	Функциональное / нефункциональное требование

ВВЕДЕНИЕ

Цель работы — спроектировать архитектуру программного обеспечения АИС фирмы по сборке и продаже компьютеров и комплектующих (вариант 34). Роль исполнителя — архитектор программного обеспечения. Результат включает состав модулей, интерфейсы, представления взаимодействия и размещения, модель хранения и способы проверки архитектурных решений.

Система поддерживает продажу отдельных комплектующих и компьютеров с услугой сборки. Основные архитектурные задачи: проверка совместимости компонентов, согласование заказа и складского резерва, сохранение исторических цен, обработка оплаты и контроль этапов комплектации, сборки, тестирования и передачи заказа.

Предлагается web-приложение с сервером на Go, клиентом на React и транзакционным хранилищем PostgreSQL. Архитектура описывает проектируемое решение; развертывание и проведение нагрузочных испытаний в состав выполненной работы не входят.

1 Архитектура программного обеспечения

1.1 Исходные требования и архитектурные ограничения

Архитектура охватывает каталог, конфигуратор ПК, корзину, заказы, склад, сборочный участок и адаптеры оплаты, доставки и уведомлений. Банк и транспортная компания являются внешними участниками. Все операции доступны через браузер с учетом роли пользователя.

Таблица 2 – Архитектурно значимые требования

Требования	**Архитектурное решение**
FR-04–FR-05; BRULE-04	Единые правила совместимости в Configurator; повторная проверка перед подтверждением заказа.
FR-07–FR-09; BRULE-01–BRULE-03	Создание заказа, фиксация цены и резервирование остатков в одной транзакции PostgreSQL.
FR-10–FR-11	Адаптер платежей, уникальный идентификатор события и идемпотентная обработка callback.
FR-13–FR-16; BRULE-05–BRULE-07	Задание на сборку и контролируемые переходы состояния; готовность собранного ПК только после успешного теста.
FR-17–FR-18	Изолированные адаптеры доставки и уведомлений с учетом ошибок внешних систем.
NFR-09–NFR-12	Отсутствие критичного состояния в памяти процесса, резервные копии, структурированные логи и конфигурация среды.

Принятые допущения: первая версия размещается на одном VPS; обязательная инфраструктура состоит из Nginx, Go backend и PostgreSQL. Конкретные провайдеры интеграций и норматив срока удержания неоплаченного резерва задаются при внедрении. Произвольное фиксированное время резерва в бизнес-правила не добавляется.

1.2 Структурные элементы ПО и их интерфейсы

Серверное приложение разделяется по функциональным областям. Внутри одного Go-процесса модули взаимодействуют через вызовы интерфейсов и прикладных сервисов. Это позволяет позже выделить отдельный модуль в самостоятельный сервис без переработки всей системы, если появится подтвержденная необходимость в независимом масштабировании.

Таблица 3 – Реестр модулей

Модуль	**Назначение**	**Зависимости**
Auth / Users	Регистрация, вход, обновление токенов, роли и права доступа.	HTTP middleware; PostgreSQL
Catalog	Каталог комплектующих, категории, характеристики, цены и признаки доступности.	Используется Configurator, Inventory и Orders
Configurator	Формирование конфигурации ПК, проверка совместимости и расчет текущей стоимости.	Читает Catalog; передает результат в Orders
Inventory	Остатки комплектующих и резервирование позиций под оформляемый заказ.	PostgreSQL; вызывается Orders
Orders	Корзина, оформление заказа, позиции заказа, итоговая цена, статусы и история изменения заказа.	Catalog, Inventory, внешние адаптеры
Payment adapter	Изоляция интеграции с платежным провайдером и обработка callback/webhook.	HTTPS API; Orders
Delivery adapter	Создание отправления и получение статуса доставки.	HTTPS API; Orders
Notification adapter	Отправка транзакционных уведомлений пользователю.	HTTPS/SMTP; Orders

1.2.1 Модуль Auth / Users

Таблица 4 – Профиль модуля Auth / Users

Аспект	**Описание**
Назначение	Регистрация, вход, обновление токенов, роли и права доступа.
Псевдоинтерфейс	register(), login(), refresh(), getUser(), changeRole()
Структуры данных	User, Role, RefreshSession
Связи	HTTP middleware; PostgreSQL

Аспект	Описание
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.2 Модуль Catalog

Таблица 5 – Профиль модуля Catalog

Аспект	Описание
Назначение	Каталог комплектующих, категории, характеристики, цены и признаки доступности.
Псевдоинтерфейс	listProducts(), getProduct(), createProduct(), updateProduct()
Структуры данных	Product, Category, Attribute, Price
Связи	Используется Configurator, Inventory и Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.3 Модуль Configurator

Таблица 6 – Профиль модуля Configurator

Аспект	Описание
Назначение	Формирование конфигурации ПК, проверка совместимости и расчет текущей стоимости.
Псевдоинтерфейс	addItem(), removeItem(), validateCompatibility(), calculatePrice()
Структуры данных	PCConfiguration, ConfigurationItem, CompatibilityRule
Связи	Читает Catalog; передает результат в Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.4 Модуль Inventory

Таблица 7 – Профиль модуля Inventory

Аспект	Описание
Назначение	Остатки комплектующих и резервирование позиций под оформляемый заказ.

Аспект	Описание
Псевдоинтерфейс	getAvailable(), reserve(), release(), adjustStock()
Структуры данных	StockItem, Reservation, StockMovement
Связи	PostgreSQL; вызывается Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.5 Модуль Orders

Таблица 8 – Профиль модуля Orders

Аспект	Описание
Назначение	Корзина, оформление заказа, позиции заказа, итоговая цена, статусы и история изменения заказа.
Псевдоинтерфейс	createCart(), checkout(), cancelOrder(), getOrder(), changeStatus()
Структуры данных	Cart, Order, OrderItem, OrderStatus
Связи	Catalog, Inventory, внешние адаптеры
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.6 Модуль Payment adapter

Таблица 9 – Профиль модуля Payment adapter

Аспект	Описание
Назначение	Изоляция интеграции с платежным провайдером и обработка callback/webhook.
Псевдоинтерфейс	createPayment(), handleCallback(), refund()
Структуры данных	Payment, PaymentEvent
Связи	HTTPS API; Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.7 Модуль Delivery adapter

Таблица 10 – Профиль модуля Delivery adapter

Аспект	**Описание**
Назначение	Создание отправления и получение статуса доставки.
Псевдоинтерфейс	createShipment(), getTrackingStatus()
Структуры данных	Shipment
Связи	HTTPS API; Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

1.2.8 Модуль Notification adapter

Таблица 11 – Профиль модуля Notification adapter

Аспект	**Описание**
Назначение	Отправка транзакционных уведомлений пользователю.
Псевдоинтерфейс	notifyOrderStatus(), notifyPayment()
Структуры данных	Notification
Связи	HTTPS/SMTP; Orders
Декомпозиция	Подмодуль единого серверного приложения. HTTP-обработчики вызывают прикладной сервис; доступ к данным скрыт за интерфейсом репозитория.

Комплектация, сборка и тестирование реализуются прикладными сценариями Orders. Для заказа с услугой сборки хранится AssemblyTask; журнал статусов и результат теста позволяют проверить допустимость перехода в состояние «Готов». Общие технические механизмы — конфигурация, логирование, аудит и проверка прав — используются всеми модулями.

1.3 Компоненты и соединители во время выполнения

Внешний интерфейс системы реализуется как web-приложение. Клиентская часть работает в браузере, серверная часть предоставляет REST API, а постоянные данные хранятся в PostgreSQL. Интеграция с оплатой, доставкой и уведомлениями выполняется через отдельные адаптеры, чтобы доменная логика не зависела от конкретного поставщика внешнего сервиса.

Таблица 12 – Компоненты и соединители

Компонент	**Назначение**	**Соединитель / интерфейс**	**Совместно используемые данные**
Браузер / React	Интерфейс пользователя	HTTPS/JSON	Каталог, корзина, заказ, учетная запись
Nginx	Единая внешняя точка входа, TLS и reverse proxy	HTTPS 443; / -> frontend; /api -> backend	Не хранит бизнес-данные
Go backend	REST API, авторизация, бизнес-правила и транзакции	HTTP внутри Docker-сети; вызовы пакетов	Доменные сущности и DTO
PostgreSQL 16	Постоянное транзакционное хранилище	SQL через pgx/v5	Пользователи, товары, остатки, конфигурации, заказы, статусы интеграций
Платежный провайдер	Оплата и возвраты	HTTPS API + webhook	Идентификатор платежа, сумма, статус
Служба доставки	Создание отправлений и трекинг	HTTPS API	Отправление, трек-номер, статус
Провайдер уведомлений	E-mail/SMS уведомления	HTTPS API или SMTP	Адресат, шаблон, статус отправки

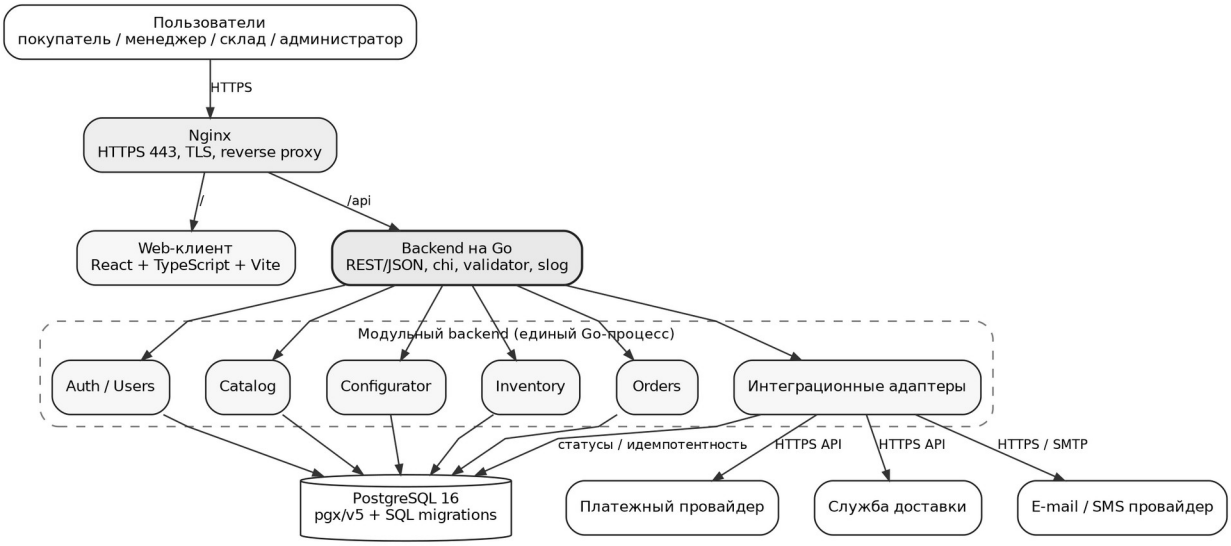


Рисунок 1 – Логическая компонентная схема программной системы

Выполнение HTTP-запросов допускается параллельно. Общим ресурсом является PostgreSQL: согласование конкурентных изменений выполняется транзакциями и блокировками. Сетевой вызов платежного провайдера производится после фиксации локального заказа, чтобы ожидание внешней системы не удерживало блокировки склада.

1.4 Выбор архитектуры и архитектурные представления

Архитектура программного обеспечения разработана с учетом требований предметной области. Для первой версии системы выбран модульный монолит: все основные бизнес-функции реализуются в одном серверном приложении на Go, но разделяются на независимые пакеты с четкими интерфейсами. Такой подход позволяет сохранить простоту развертывания и транзакционную целостность, не создавая преждевременную инфраструктурную сложность микросервисов.

Таблица 13 – Сравнение архитектурных вариантов

Вариант	**Преимущества**	**Ограничения**
Монолит без модульных границ	Простой запуск и единая БД	Сильная связанность функций и сложность локальных изменений.
Модульный монолит — выбран	Единая транзакционная граница, интерфейсы модулей, простое развертывание	Модули поставляются общей версией; требуется контроль зависимостей.
Микросервисы	Независимое масштабирование и поставка	Распределенные транзакции, сетевые отказы и более сложная эксплуатация.

Выбранная архитектура сочетает клиент-серверную структуру и многоуровневое построение серверного приложения. На уровне представления находится React-клиент, на транспортном уровне – REST API, далее располагаются прикладные сервисы и доменная логика, а доступ к данным выполняется через репозитории PostgreSQL. Интеграции с внешними системами вынесены в адаптеры.

Таблица 14 – Архитектурные представления

Представление	**Содержание**
Модульное	Auth, Catalog, Configurator, Inventory, Orders и интеграционные адаптеры.
Компонентов и соединителей	Браузер → Nginx → Go backend → PostgreSQL; backend → внешние HTTPS API.
Размещения	Один Linux VPS, контейнеры, внутренняя сеть и отдельный постоянный том БД.
Данных	Исторические цены заказа, остатки и резервы, платежные события и история состояний.

1.5 Структуры распределения и размещения

Клиент работает в браузере. Nginx завершает TLS, выдает собранный frontend и направляет /api к Go backend. Backend обращается к PostgreSQL по внутренней Docker-сети. Порт БД не публикуется в Интернет. Единственный VPS является общей точкой отказа; высокая доступность требует отдельного расширения инфраструктуры.

Таблица 15 – Размещение и эксплуатационные элементы

Компонент	**Размещение**	**Режим работы / интерфейс**	**Данные и эксплуатационные требования**
Nginx	VPS, контейнер или host service	80/443 наружу; TLS; reverse proxy	Сертификаты Let's Encrypt, конфигурация Nginx
Frontend	Docker container	Статическая production-сборка React	Образ из GHCR; без постоянного состояния
Go backend	Docker container	REST API; внутренний порт, например 8080	Образ из GHCR; restart unless-stopped; /healthz и /readyz
PostgreSQL 16	Docker container	Только внутренняя Docker-сеть	Named volume на SSD; отдельный пользователь БД
Миграции	One-shot container / команда backend	Выполняются перед запуском новой версии	Версионируемые SQL из репозитория
Резервное копирование	cron / отдельный контейнер	Ночной pg_dump	Копия хранится вне VPS; периодическая проверка восстановления
Логи	stdout/stderr контейнеров	JSON от slog	Просмотр через docker logs; позднее возможна отправка в Loki/аналог
Секреты	.env на VPS / secret store	Не включаются в image и Git	Права доступа только у пользователя развертывания

1.5.1 Технологический стек и файловая структура

Для реализации выбран следующий технологический стек. Указанные версии являются целевыми версиями проекта, а не результатом проверки актуальности пакетов.

Таблица 16 – Технологический стек

Уровень	**Технология**	**Назначение и обоснование**
Backend	Go 1.25.5	Основной язык серверной части; статическая типизация, простое развертывание одним бинарным файлом, удобная конкурентность.
HTTP API	net/http + go-chi/chi	REST/JSON API, маршрутизация, middleware, разделение обработчиков по функциональным областям.
Валидация	go-playground/validator/v10	Проверка входных DTO до вызова бизнес-логики.
Конфигурация	cleanenv + переменные окружения	Разделение local/stage/prod-конфигурации; секреты не хранятся в репозитории.
Логирование	log/slog	Структурированные JSON-логи в рабочей среде, текстовый формат локально.
Аутентификация	golang-jwt/jwt/v5 + golang.org/x/crypto	Access/refresh JWT, хеширование паролей, middleware проверки ролей.
База данных	PostgreSQL 16 + pgx/v5	Транзакционное хранение пользователей, каталога, остатков, заказов, платежных и логистических статусов.
Идентификаторы	google/uuid	UUID для внешне видимых сущностей и событий интеграций.
Миграции	Версионируемые SQL-файлы	Изменения схемы БД хранятся вместе с кодом и выполняются перед запуском новой версии.
Тестирование	testing + Testify + httpexpect	Unit-тесты бизнес-логики и HTTP-обработчиков, интеграционные тесты с PostgreSQL.
Frontend	React 19 + TypeScript 5.8 + Vite 7	SPA-интерфейс; типизированный клиент, быстрая сборка и отдельный production bundle.
HTTP-клиент frontend	Axios + React Router	Вызов REST API и клиентская маршрутизация.
Контейнеризация	Docker + Docker Compose	Одинаковая среда локальной разработки и развертывания; multi-stage Dockerfile для Go.

Уровень	Технология	Назначение и обоснование
CI/CD	GitHub + GitHub Actions + GHCR	Pull Request, автоматические проверки, сборка Docker-образов и доставка на сервер.
Edge / TLS	Nginx + Let's Encrypt	HTTPS, reverse proxy, выдача статического frontend и проксирование /api к Go-приложению.

```
cmd/app/main.go
internal/config/
internal/http/handlers/
internal/http/middleware/
internal/domain/
internal/service/
internal/repository/postgres/
internal/integration/
migrations/
web/
deploy/
.github/workflows/
```

Пакеты domain содержат сущности и правила; service реализует сценарии. HTTP и PostgreSQL зависят от прикладных интерфейсов. Секреты передаются конфигурацией среды. Redis и брокер сообщений не требуются для базового варианта.

1.5.2 Управление версиями и CI/CD

Исходный код размещается в GitHub. Для коллективной разработки применяется модель с защищенной основной веткой: main содержит состояние, разрешенное к развертыванию, develop используется как интеграционная ветка, а изменения выполняются в feature/* и попадают в целевую ветку только через Pull Request. Для main включаются обязательные status checks, поэтому код не может быть слит, пока автоматические проверки не завершились успешно.

Таблица 17 – Конвейер проверки и поставки

Событие	Действия	Результат
Pull Request / push в develop	Checkout; setup Go 1.25.5; кэш модулей; go mod download; проверка форматирования; go vet; golangci-lint.	Статус проверки backend
Pull Request / push в develop	go test ./... с race detector и покрытием; интеграционные тесты с PostgreSQL service container.	Отчет тестов / coverage

Событие	**Действия**	**Результат**
Pull Request / push в develop	Node setup; npm ci; npm run lint; npm run build.	Проверенная production-сборка frontend
Pull Request	docker build backend и frontend без публикации.	Гарантия, что Dockerfile собирается
Merge / push в main	Сборка Docker-образов backend и frontend, теги sha-<commit> и latest, публикация в GHCR.	Версионированные образы
После публикации образов	SSH на VPS; docker compose pull; выполнение миграций; docker compose up -d; healthcheck.	Новая версия в рабочей среде
Ошибка healthcheck	Фиксация неуспешного deploy; возврат на предыдущий известный тег образа.	Контролируемый откат

Образы публикуются с неизменяемым тегом commit SHA. Перед миграцией проверяется резервная копия и совместимость схемы со старой версией. Откат образа применим только при совместимой схеме; разрушающие миграции требуют отдельного плана восстановления.

1.6 Модель хранения данных и инварианты

Таблица 18 – Основные сущности хранения

Группа	**Сущности и связи**
Пользователи	User, Role, RefreshSession; пользователь имеет роли и сессии.
Каталог и конфигурации	Product → Category; ProductAttribute; PCConfiguration → ConfigurationItem → Product.
Склад	StockItem связан с Product и складом; Reservation связан с Order; движения фиксирует StockMovement.
Заказы	Order → OrderItem; каждая позиция хранит количество и цену-снимок. OrderStatusHistory сохраняет переходы.
Исполнение	AssemblyTask → Order; хранит исполнителя, состояние и результат тестирования.
Интеграции	Payment и PaymentEvent → Order; Shipment → Order; Notification хранит состояние отправки.

При хранении физического остатка quantity и активного резерва reserved выполняются инварианты: quantity ≥ 0; reserved ≥ 0; reserved ≤ quantity; available = quantity − reserved. При выдаче товара физический остаток и резерв уменьшаются согласованно. При отмене освобождается только активный резерв, повторное освобождение запрещается.

Исторический состав и стоимость заказа не пересчитываются из текущего каталога. Количество позиции положительно; сумма вычисляется из сохраненных цен. Для идентификатора платежа и внешнего события действует уникальность в пределах провайдера. Оплаченный или выполненный заказ архивируется без физического удаления.

1.7 Сквозные сценарии и согласование транзакций

Таблица 19 – Последовательности выполнения

Сценарий	**Последовательность и граница согласованности**
Заказ комплектующих	Проверка корзины → блокировка остатков → фиксация цен и заказа → резерв → COMMIT → создание платежа.
Заказ ПК со сборкой	Проверка совместимости → оформление и резерв → подтвержденная оплата → задание складу и сборщику → успешный тест → готовность.
Подтверждение оплаты	Проверка подлинности события → блокировка платежа → дедупликация → сверка суммы и заказа → обновление платежа и статуса → COMMIT.
Отмена	Проверка полномочий и состояния → освобождение активного резерва → запись состояния и причины → отдельная операция возврата при необходимости.
Доставка	Проверка готовности → создание отправления через адаптер → сохранение внешнего идентификатора → отслеживание → подтверждение вручения.

При неопределенном результате внешнего вызова операция помечается как требующая сверки. Повтор создания платежа использует устойчивый идентификатор запроса. Ошибка уведомления не отменяет уже выполненную оплату или резервирование; сообщение хранится для повторной отправки.

1.8 Алгоритмы критических операций

1.8.1 Атомарное резервирование

```
BEGIN
проверить идемпотентный ключ оформления;
заблокировать строки остатков по product_id в одном порядке;
проверить доступность всех позиций и совместимость;
создать Order и OrderItem с ценами-снимками;
увеличить reserved и создать Reservation;
записать начальный статус и результат операции;
COMMIT
при ошибке любой позиции: ROLLBACK;
```

Одинаковый порядок захвата блокировок снижает риск взаимной блокировки. Ограничения БД дополняют проверку сервиса. При конфликте транзакции допустим ограниченный повтор всей операции с тем же ключом оформления.

1.8.2 Идемпотентный платежный callback

```
проверить подпись и идентификатор провайдера;
BEGIN
получить платеж и заказ с блокировкой;
если event_id уже обработан: вернуть сохраненный результат;
проверить сумму, валюту и допустимость перехода;
сохранить событие и изменить состояние платежа;
обновить заказ только при выполнении условий;
COMMIT
вернуть подтверждение приема события;
```

Уникальное ограничение на пару «провайдер, event_id» защищает от одновременного получения одинаковых событий. Платеж по отмененному заказу направляется на сверку; автоматическое создание задания на сборку в таком случае запрещено.

1.9 Контракты внешних интерфейсов

Предлагаемые маршруты описывают контракт проектируемого API. Клиент передает данные через HTTPS/JSON; сервер проверяет DTO, полномочия и бизнес-правила. Ошибка возвращается с кодом, понятным сообщением и идентификатором запроса.

Таблица 20 – Основные HTTP-контракты

Операция	**Назначение**	**Доступ**
GET /api/products	Каталог и фильтрация	Покупатель / гость
POST /api/configurations/validate	Проверка совместимости	Покупатель
POST /api/orders	Оформление, резерв и цена-снимок	Покупатель; Idempotency-Key
GET /api/orders/{id}	Статус и состав заказа	Владелец / уполномоченный сотрудник
POST /api/orders/{id}/cancel	Отмена по правилам состояния	Владелец / сотрудник с правом
POST /api/payments/webhook	Прием подписанного события	Платежный провайдер
POST /api/assembly/{id}/test	Сохранение результата теста	Сборщик
POST /api/stock/movements	Приход или списание с причиной	Склад
GET /healthz; GET /readyz	Работоспособность и готовность	Эксплуатационный контур

Пример оформления: {"items":[{"productId":"uuid","qty":1}], "assemblyRequired":true, "configurationId":"uuid", "deliveryMethod":"pickup"}. Цена не принимается как доверенное значение от клиента. Ответ содержит orderId, итоговую сумму и состояние; при повторе с тем же ключом возвращается тот же заказ.

1.10 Безопасность, надежность и эксплуатация

Таблица 21 – Разграничение доступа

Роль	**Разрешенные действия**
Покупатель	Каталог, конфигурации, собственные заказы и допустимая отмена.
Менеджер	Управление каталогом и сопровождение заказов в пределах полномочий.
Склад	Остатки, приход, списание, комплектация и журнал движений.
Сборщик	Назначенные задания, сборка и результаты тестирования.
Администратор	Учетные записи сотрудников, роли и системные настройки.

Backend проверяет роль и принадлежность объекта на каждом защищенном маршруте. Пароли хранятся как стойкие хеши, секреты — вне Git и контейнерных образов. Access/refresh токены проверяются по сроку и подписи; refresh-сессии допускают отзыв. Реквизиты карты обрабатывает внешний платежный провайдер.

Журналы slog содержат requestId, orderId, операцию и результат без паролей, токенов и реквизитов. Контролируются ошибки API, задержка запросов, конфликты резерва, повторы callback и ошибки интеграций. Технический лог дополняет отдельную историю критических действий.

Резервная копия PostgreSQL создается не реже раза в сутки и сохраняется вне VPS. Периодически выполняется пробное восстановление в изолированной среде с проверкой заказов, связей и платежных состояний. Суточное копирование само по себе не гарантирует нулевую потерю данных; RPO и RTO уточняются при внедрении.

1.11 Прослеживаемость и проверки

Таблица 22 – Архитектурные проверки

Требования	**Проверка и ожидаемый результат**
FR-05; BRULE-04	Несовместимые сокеты или тип памяти: заказ ПК не подтверждается.
FR-08; BRULE-02	Изменить цену в каталоге после оформления: цена заказа остается прежней.
FR-09; BRULE-03	Два заказа на последний экземпляр: резерв успешно создается только для одного.
FR-11	Одновременный повтор callback: единственное изменение платежа и заказа.
FR-13–FR-15; BRULE-06	Отрицательный тест ПК: состояние «Готов» не устанавливается.
FR-12; BRULE-07	Отмена после начала сборки без полномочий: отказ без освобождения резерва.
FR-18	Отключить уведомления: заказ и оплата сохраняются; отправка доступна для повтора.
NFR-09	Повтор запроса через другой экземпляр backend: постоянное состояние доступно из БД.
NFR-10	Восстановить копию и проверить целостность заказов, позиций и резервов.
BR-03	Покупатель запрашивает чужой заказ: отказ в доступе.

Перечисленные сценарии являются планом верификации архитектуры. Фактические результаты тестов и показатели производительности фиксируются после реализации и испытаний.

1.12 Выводы по архитектуре

Определены восемь основных модулей, их контракты и структуры данных. Выбран модульный монолит с многоуровневым построением backend. Создание заказа и резервирование выполняются в единой транзакционной границе; внешние интеграции изолированы адаптерами. Предусмотрены исторические цены, проверка состояний, аудит, резервное копирование и контролируемая поставка.

2 Краткое описание хода выполнения

В ходе работы выделены архитектурно значимые сценарии продажи комплектующих и сборки ПК. Определены модульные границы, интерфейсы и модель взаимодействия компонентов. Сопоставлены варианты архитектуры и выбран модульный монолит. Затем уточнены размещение, технологический стек, сущности хранения и инварианты транзакций.

Наиболее существенными решениями стали атомарное резервирование при конкурентных заказах, сохранение исторической стоимости, идемпотентная обработка платежа и контроль готовности ПК по результату теста. Для проверки этих решений сформирован набор наблюдаемых сценариев, а для эксплуатации — регламент поставки, резервирования и восстановления.

ЗАКЛЮЧЕНИЕ

Разработана архитектура ПО АИС фирмы по сборке и продаже компьютеров и комплектующих для варианта 34. Описаны модульная структура, компоненты и соединители, размещение и хранение данных. Проект включает интерфейсы модулей и API, критические алгоритмы, механизмы безопасности, CI/CD и план архитектурных проверок.

Выбранный стек Go, React, PostgreSQL и Docker Compose позволяет реализовать единую транзакционную систему с разделенными обязанностями модулей. Архитектура допускает запуск нескольких экземпляров backend и последующее выделение отдельных сервисов при подтвержденной необходимости.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Основы программной инженерии: методические указания к выполнению лабораторных работ / сост. П. А. Охтилев, А. Э. Зянчурин. Санкт-Петербург: ГУАП, 2021. 47 с.

2. Фаулер М. UML. Основы: краткое руководство по стандартному языку объектного моделирования. 3-е изд. Санкт-Петербург: Символ, 2014. 192 с.

ПРИЛОЖЕНИЕ А

Контроль переходов заказа

Таблица 23 – Переходы для заказа с услугой сборки

Событие	**Условие**	**Результат**
Оформление	Цена зафиксирована, резерв создан	Ожидает оплаты
Подтверждение оплаты	Событие проверено, резерв действителен	Оплачен; задание на сборку
Комплектация	Склад подготовил полный состав	Скомплектован
Начало сборки	Назначен сборщик	В сборке
Завершение сборки	Комплектность подтверждена	На тестировании
Тест успешен	Сохранен положительный результат	Готов
Тест неуспешен	Сохранено описание дефекта	Доработка и повторное тестирование
Выдача / доставка	Заказ готов и получатель подтвержден	Выдан / передан в доставку
Отмена	Проверены состояние и полномочия	Отменен; резерв освобожден один раз

Заказ отдельных комплектующих проходит комплектацию и проверку готовности к выдаче без задания на сборку ПК. Возврат оплаты фиксируется самостоятельной платежной операцией.

ПРИЛОЖЕНИЕ Б

Таблица 24 – Архитектурные риски

Риск	**Способ предотвращения**
Двойной резерв	Блокировки, ограничения БД и единая транзакция.
Изменение цены задним числом	Цена-снимок в OrderItem.
Повторная оплата по callback	Уникальный event_id и проверка перехода.
Выдача непроверенного ПК	Проверка успешного результата тестирования.
Отказ внешнего сервиса	Тайм-аут, учет неопределенного результата и сверка.
Отказ VPS	Внешние резервные копии и проверка восстановления.
Несовместимая миграция	Поэтапные изменения схемы и план отката.
Утечка заказа	RBAC и проверка владельца объекта.